

Débuter avec R



langage de programmation



**environnement de développement
intégré**

Un panorama de Positron

File Edit Selection View Go Run Terminal Help

Manipulations.R - IATA_EVA_challenge - Positron

New Open

Search

R 4.6.1 IATA_EVA_challenge

Manipulations.R

Data: marathon

home > lbelzile > Documents > Dropbox > Enseignement > MATH60602 > Manipulations.R > ...

```
72 top25 <- berlin %>%
73   mutate(position = as.numeric(position), km = as.numeric(km))
74
75 library(tinyplot)
76 tinytheme("clean2")
77 plt(
78   x = speed ~ km | position,
79   type = "l",
80   xlab = "km",
81   ylab = "",
82   main = "Performance des 25 meilleurs coureurs",
83   sub = "Vitesse (km/h) par segment de 5km",
84   cap = "Source: Marathon de Berlin, 2022",
85   data = top25
86 )
87
88 
```

CONSOLE

TERMINAL

PROBLEMS

OUTPUT

PORTS

DEBUG CONSOLE

~/Documents/Dropbox/IATA_EVA_challenge

```
+ mapping = aes(
+   x = km,
+   group = factor(position),
+   col = factor(position),
+   y = speed
+ )
+ ) +
+   geom_line(show.legend = FALSE) +
+   labs(
+     x = "km",
+     y = "",
+     # col = "rang",
+     subtitle = "Vitesse (km/h) par segment de 5km",
+     title = "Performance des 25 meilleurs coureurs",
+     caption = "Source: Marathon de Berlin, 2022",
+   )
+ > ggplot2:::them
```

SESSION

CONNECTIONS

HELP

HISTORY

VIEWER

VARIABLES

DATA

TESTS

PLOTS

R 4.6.1 Manipulations.R

ggplot2 line chart 7

Performance des 25 meilleurs coureurs
Vitesse (km/h) par segment de 5km

Source: Marathon de Berlin, 2022

plot 14

ggplot2 line ch...

Ln 116, Col 1 Spaces: 2 UTF-8 LF R Sign In 4 / 54

Console



```
CONSOLE  TERMINAL  PROBLEMS  OUTPUT  PORTS  DEBUG CONSOLE
~/Documents/Dropbox/IATA_EVA_challenge

+ mapping = aes(
+   x = km,
+   group = factor(position),
+   col = factor(position),
+   y = speed
+ )
+ ) +
+ geom_line(show.legend = FALSE) +
+ labs(
+   x = "km",
+   y = "",
+   # col = "rang",
+   subtitle = "Vitesse (km/h) par segment de 5km",
+   title = "Performance des 25 meilleurs coureurs",
+   caption = "Source: Marathon de Berlin, 2022",
+ )
> ggplot2::theme
```

R est en attente de vos instructions

Tapez votre code, et appuyez sur Retour/Entrée pour le compiler

À votre tour

```
CONSOLE  TERMINAL  PROBLEMS  OUTPUT  PORTS  DEBUG CONSOLE
~/Documents/Dropbox/IATA_EVA_challenge
+ mapping = aes(
+   x = km,
+   group = factor(position),
+   col = factor(position),
+   y = speed
+ )
+ ) +
+ geom_line(show.legend = FALSE) +
+ labs(
+   x = "km",
+   y = "",
+   # col = "rang",
+   subtitle = "Vitesse (km/h) par segment de 5km",
+   title = "Performance des 25 meilleurs coureurs",
+   caption = "Source: Marathon de Berlin, 2022",
+ )
> ggplot2::them
```

Tapez `2 + 2` dans la console

Appuyez sur Retour

```
2 + 2
```

```
## [1] 4
```

**Ce calcul est éphémère.
Pour recompiler, il faudra retaper les instructions.**

On va plutôt enregistrer le code dans un document R

**On peut aussi assigner le résultat à une variable
pour l'utiliser subséquemment**

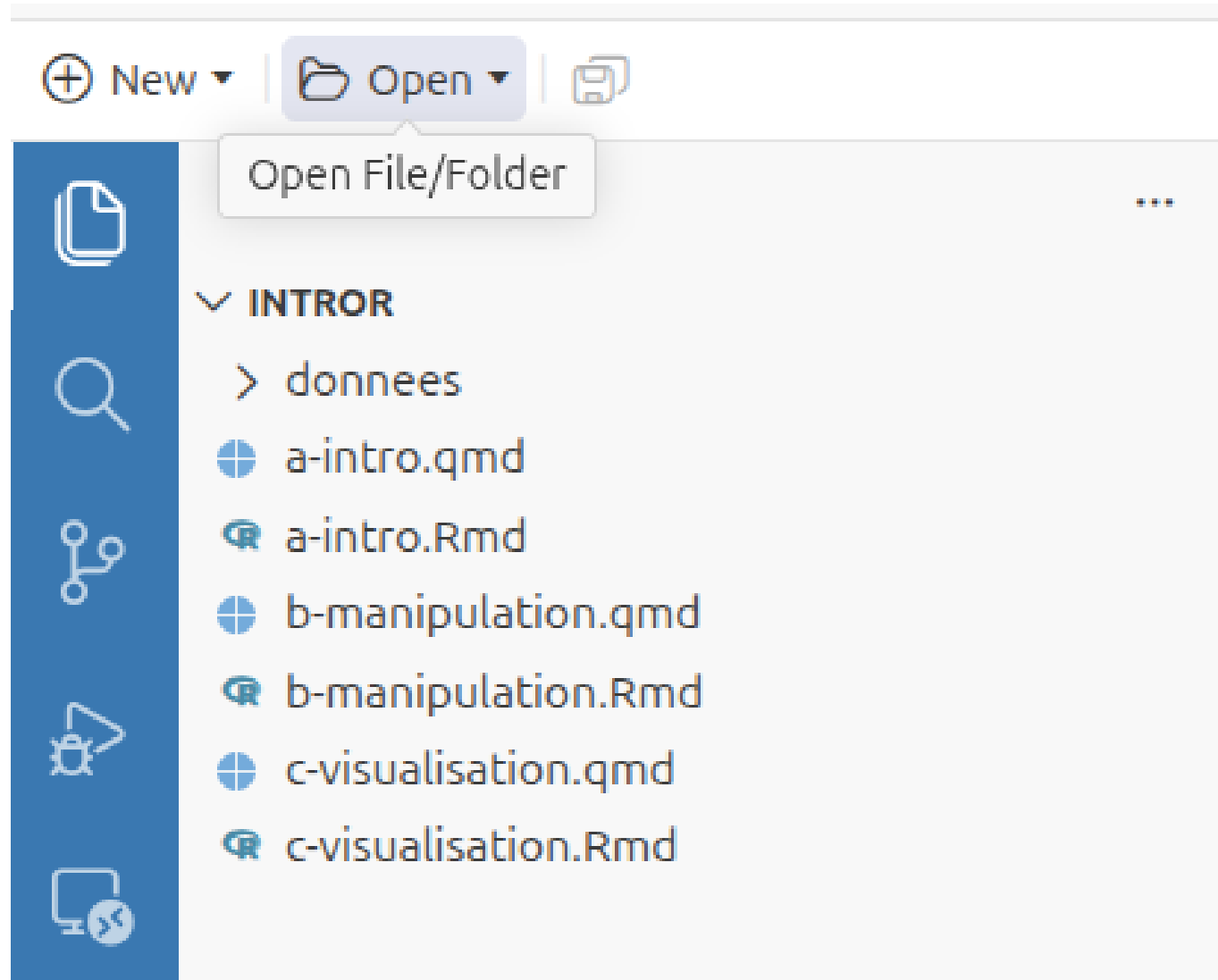
Arborescence et fichiers

R pointe vers un répertoire local

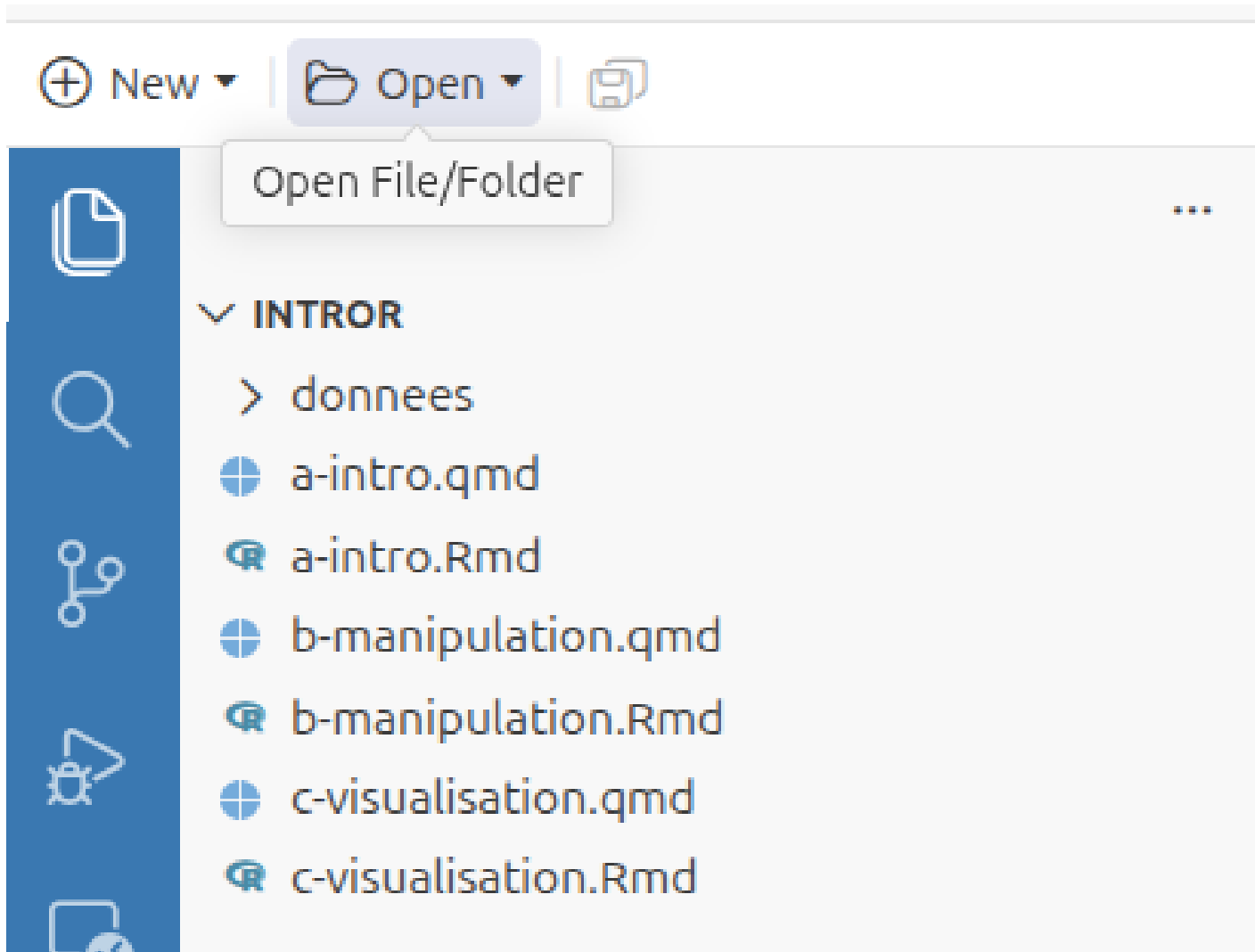
Utiliser `getwd()` pour trouver l'emplacement

Travailler avec chemin relatif

Dossiers dans Positron

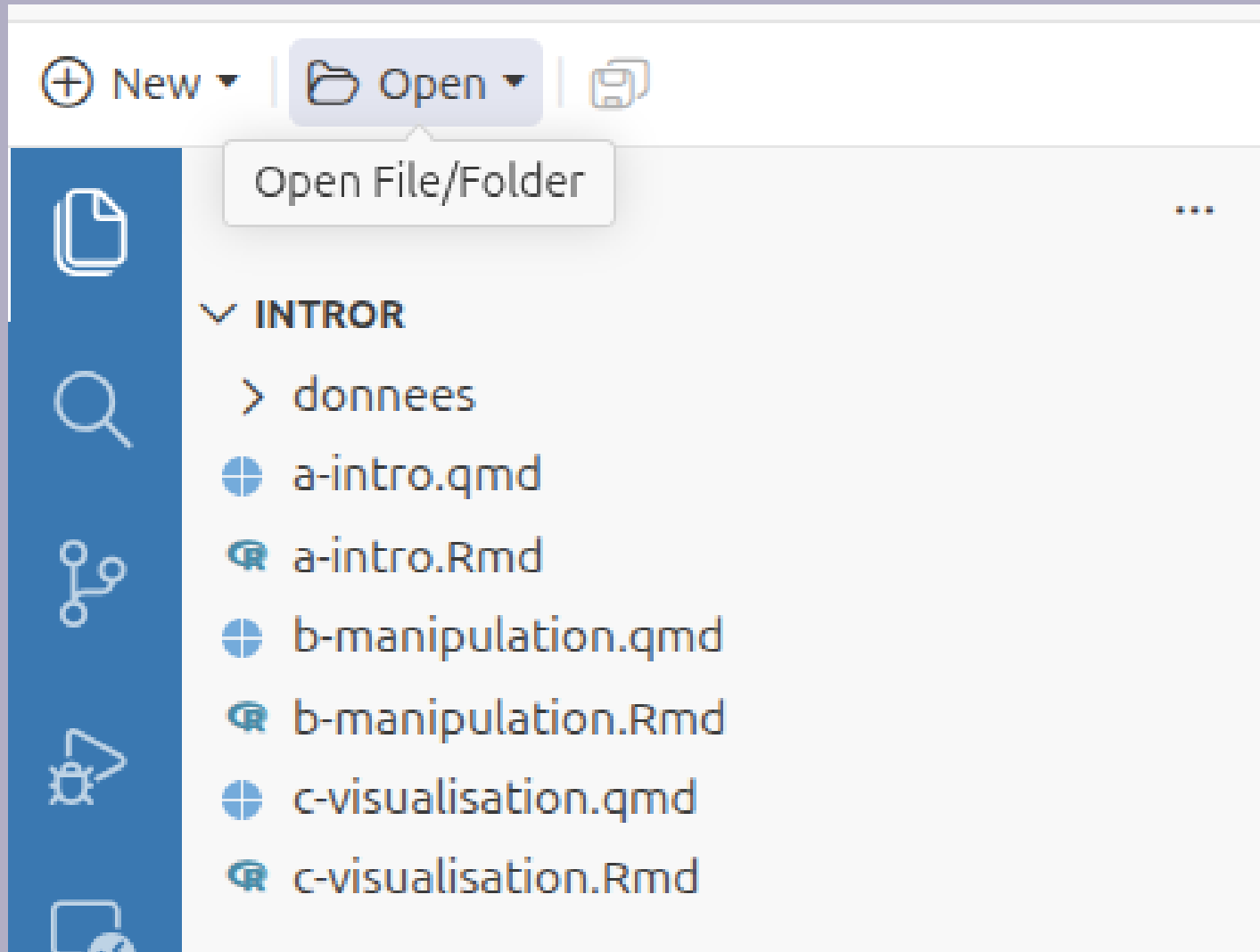


Panneau avec fichiers



**Tous les fichiers dans
votre répertoire de
travail**

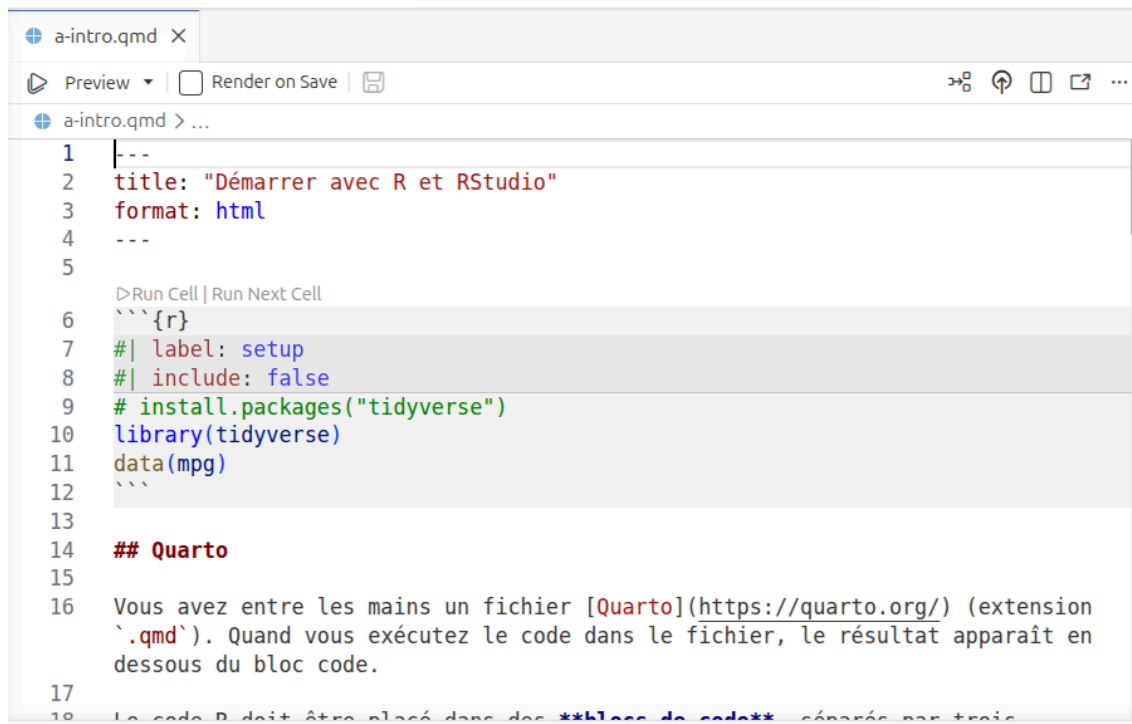
À votre tour



Trouvez `a-intro.qmd`

**Cliquez sur le nom
pour ouvrir le fichier**

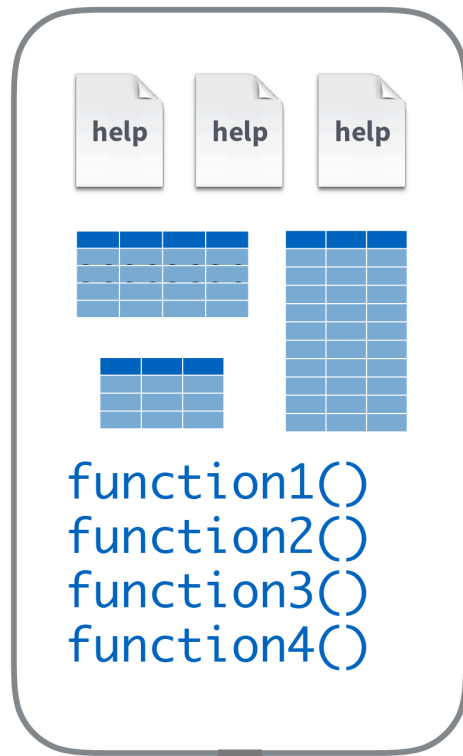
Panneau source



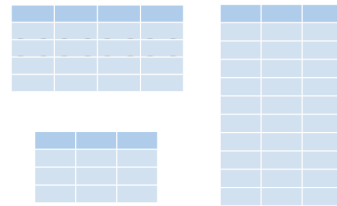
```
1 |--
2 title: "Démarrer avec R et RStudio"
3 format: html
4 ---
5
6 >Run Cell | Run Next Cell
7 ```{r}
8 #| label: setup
9 #| include: false
10 # install.packages("tidyverse")
11 library(tidyverse)
12 data(mpg)
13 ```
14 ## Quarto
15
16 Vous avez entre les mains un fichier [Quarto](https://quarto.org/) (extension
17 `.qmd`). Quand vous exécutez le code dans le fichier, le résultat apparaît en
18 dessous du bloc code.
```

Les documents
s'ouvrent ici

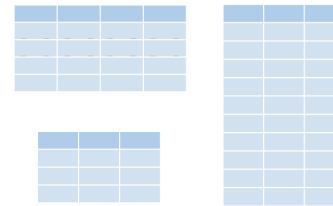
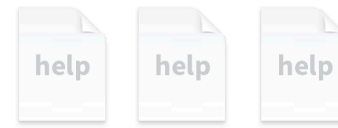
Paquets



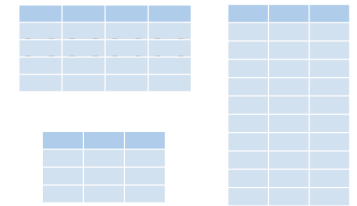
Base R



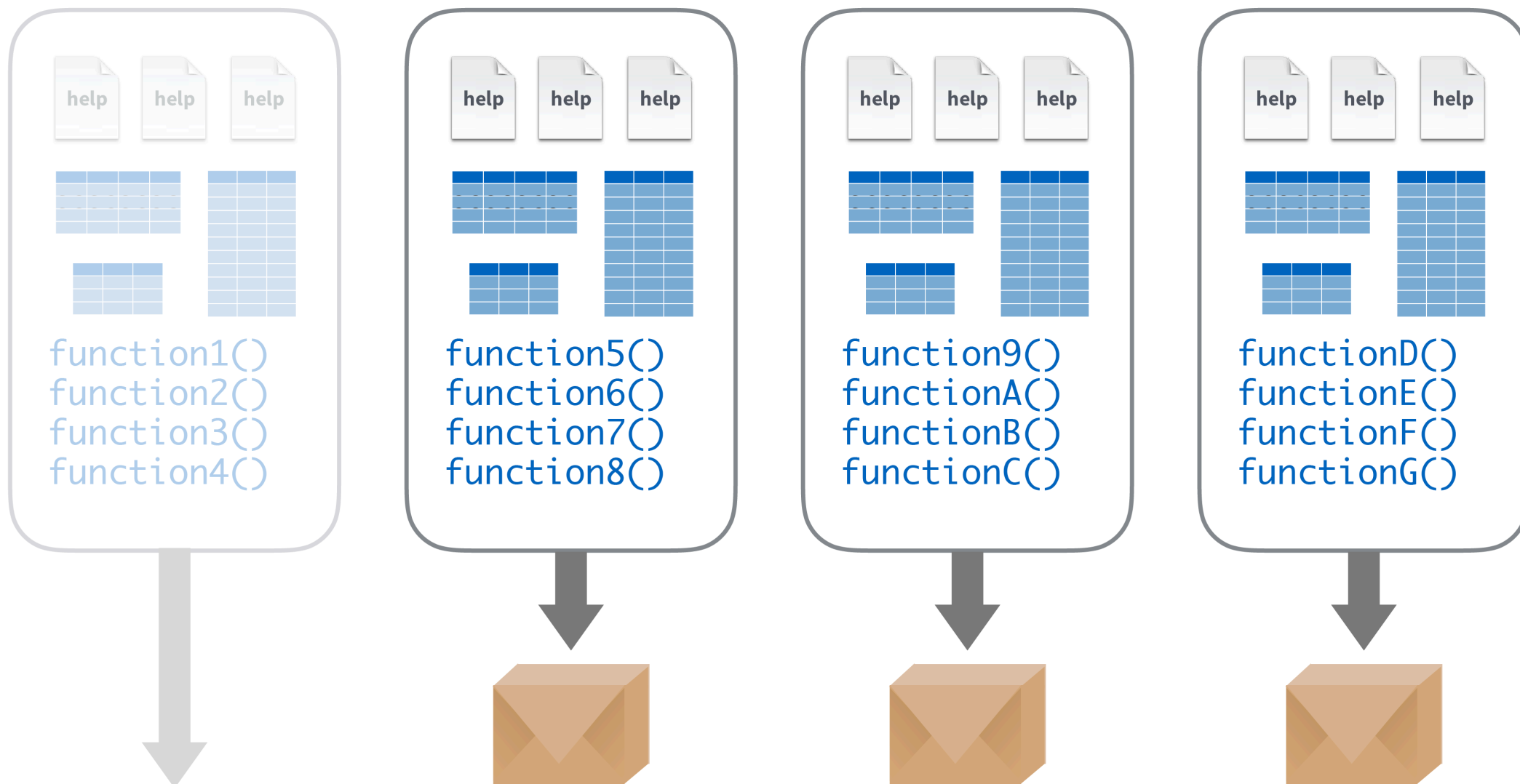
`function5()`
`function6()`
`function7()`
`function8()`



`function9()`
`functionA()`
`functionB()`
`functionC()`



`functionD()`
`functionE()`
`functionF()`
`functionG()`



Utiliser des paquets

```
install.packages("nomdupaquet")  
# Commentaire: sensible à la casse
```

**Télécharge les fonctionnalités
sur votre ordinateur.**

Une fois par ordinateur.

Fichiers téléchargés via le **CRAN.**

```
library(nomdupaquet)
```

Charge le paquet

Effectuer une fois par session

À votre tour

Installez le paquet `remotes`

Utilisez les commandes suivantes pour installer `hecmulti`:

```
remotes::install_github("lbelzile/hecmulti")
```

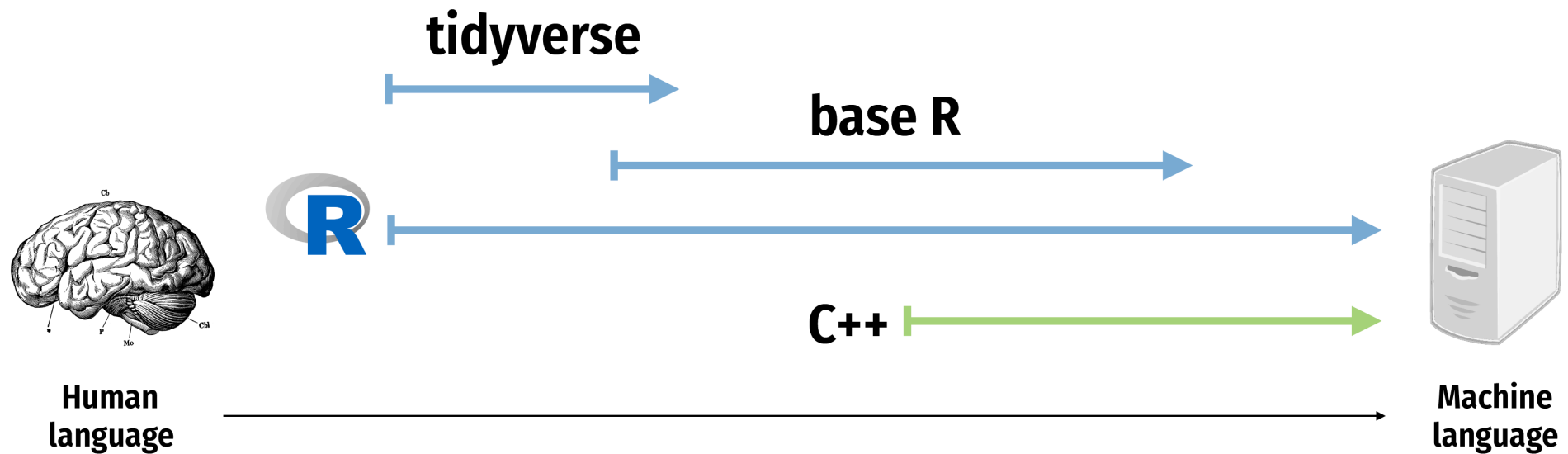
02:00

tidyverse

« Le tidyverse est une collection de paquets R conçus par des programmeurs opiniâtre pour la science des données. Tous les paquets partagent la même philosophie de conception, et ont une grammaire et une structure commune. »



tidyverse



Paquets et fonctionnalités

Charger un paquet ajoute à l'environnement toutes ses fonctions et bases de données.

Peut masquer des fonctions homonymes!

```
> library(tidyverse)
```

```
— Attaching core tidyverse packages
```

```
✓ forcats 1.0.1      ✓ tibble 3.3.1
```

```
✓ readr 2.2.0
```

```
— Conflicts
```

```
✖ nlme::collapse()      masks dplyr::collapse()
```

```
✖ lubridate::duration() masks arrow::duration()
```

```
✖ dplyr::filter()       masks stats::filter()
```

```
✖ dplyr::lag()           masks stats::lag()
```

```
✖ car::recode()          masks dplyr::recode()
```

```
✖ car::some()            masks purrr::some()
```

Fonctionnalités

Charger une base de données

```
data(bd, package = "nomdupaquet")
```

Utiliser directement une fonction sans charger un paquet

```
paquet::fonction(...)
```

À votre tour

Chargez les données `renfe` du paquet `hecmulti`

```
data(renfe, package = 'hecmulti')  
renfe
```

01:00

Quarto

The screenshot displays the Quarto editor interface. On the left, a code editor shows a file named 'a-intro.qmd' with the following content:

```
---  
title: "Démarrer avec R et RStudio"  
format: html  
---  
  
D-Run Cell | Run Next Cell  
```{r}  
#| label: setup
#| include: false
install.packages("tidyverse")
library(tidyverse)
data(mpg)
```\n\n## Quarto\n\nVous avez entre les mains un fichier [Quarto](https://quarto.org/) (extension '.qmd'). Quand vous exécutez le code dans le fichier, le résultat apparaît en dessous du bloc code.\n\nLe code R doit être placé dans des blocs de code, séparés par trois accents graves. Essayez d'exécuter ce bloc en cliquant sur le triangle vert Démarrer (*Run*) ou en appuyant sur *Ctrl+Shift+Enter* (ou *Cmd+Shift+Enter* sur Mac).
```

On the right, the rendered HTML output is shown in a viewer window. It features a teal header 'Analyse multidimensionnelle appliquée' and a table of contents. The main content is titled 'Démarrer avec R et RStudio' and 'Quarto'. It includes a paragraph explaining the Quarto workflow and a code block for a scatter plot:

```
ggplot(data = mpg) +  
  geom_point(mapping = aes(x = cty,  
                           y = hwy),  
              alpha = 0.2)
```

Below the code, a scatter plot is displayed with 'cty' on the x-axis and 'hwy' on the y-axis, showing a positive correlation between city fuel efficiency and highway fuel efficiency.

Format de document
qui combine texte et code

Un livret
pour votre analyse

Quarto

▾ ## Meet the penguins

The ``penguins`` data from the `[**palmerpenguins**](https://allisonhorst.github.io/palmerpenguins "palmerpenguins R package")` package contains size measurements for ``r nrow(penguins)`` penguins from three species observed on three islands in the Palmer Archipelago, Antarctica.

The plot below shows the relationship between flipper and bill lengths of these penguins.

Texte

Quarto

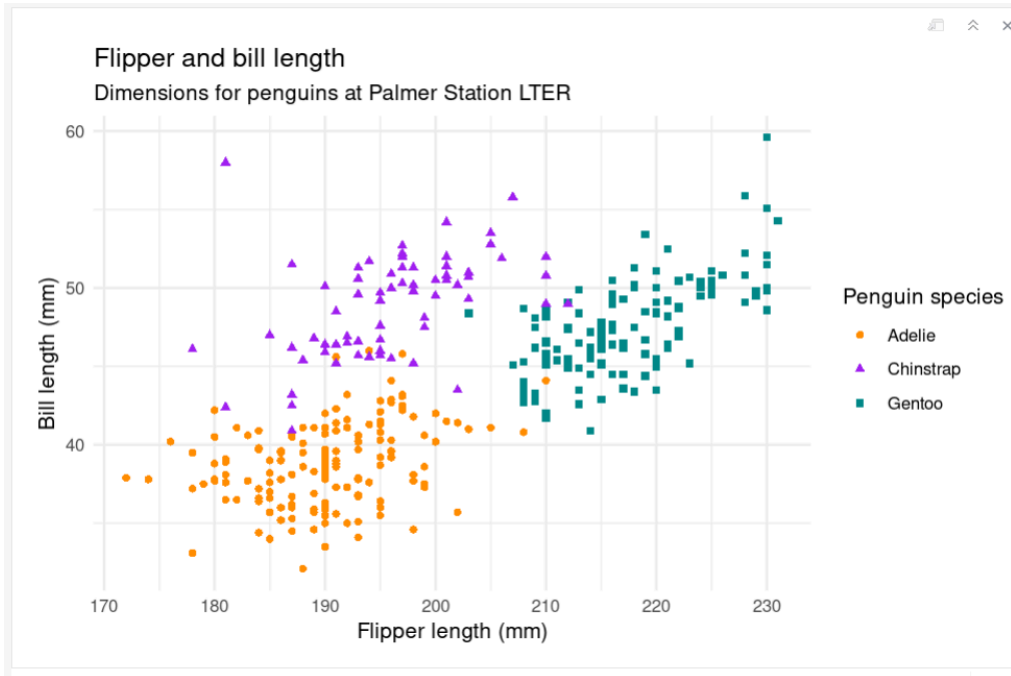
```
```{r}
#| label: plot-penguins
#| warning: false
#| echo: false

ggplot(penguins,
 aes(x = flipper_length_mm, y = bill_length_mm)) +
 geom_point(aes(color = species, shape = species)) +
 scale_color_manual(values = c("darkorange", "purple", "cyan4")) +
 labs(
 title = "Flipper and bill length",
 subtitle = "Dimensions for penguins at Palmer Station LTER",
 x = "Flipper length (mm)", y = "Bill length (mm)",
 color = "Penguin species", shape = "Penguin species"
) +
 theme_minimal()
```
```

Texte

Code

Quarto



Texte

Code

Sortie

À votre tour

```
```{r}
#| label: plot-penguins
#| warning: false
#| echo: false

ggplot(penguins,
 aes(x = flipper_length_mm, y = bill_length_mm)) +
 geom_point(aes(color = species, shape = species)) +
 scale_color_manual(values = c("darkorange", "purple", "cyan4")) +
 labs(
 title = "Flipper and bill length",
 subtitle = "Dimensions for penguins at Palmer Station LTER",
 x = "Flipper length (mm)", y = "Bill length (mm)",
 color = "Penguin species", shape = "Penguin species"
) +
 theme_minimal()
```
```

Lisez les instructions

**Compilez le bloc de code
avec le bouton Preview**

(Icône avec triangle, et Preview)

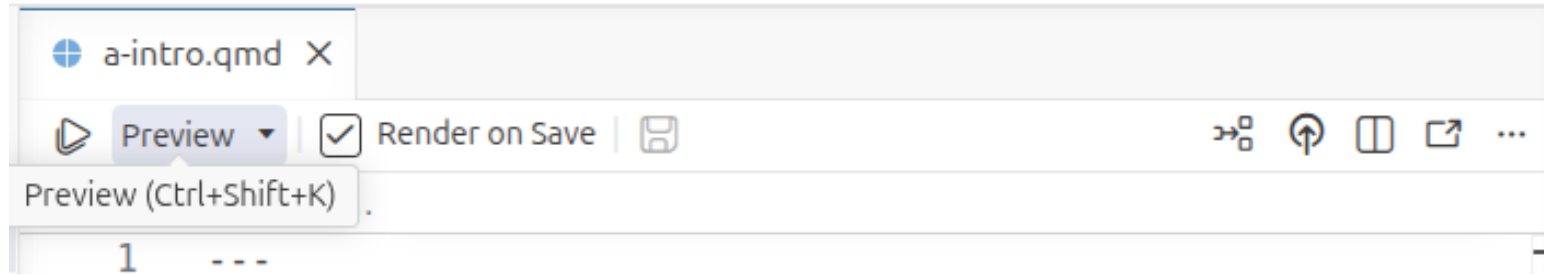
À votre tour

Ajoutez un nouveau bloc

Inscrivez $2 + 2$ et compilez

Compilation

Compiler un document Quarto



À votre tour

Quelle est la différence entre:

```
filter(mtcars, cyl == 4)
```

```
four_cyls <- filter(mtcars, cyl == 4)
```

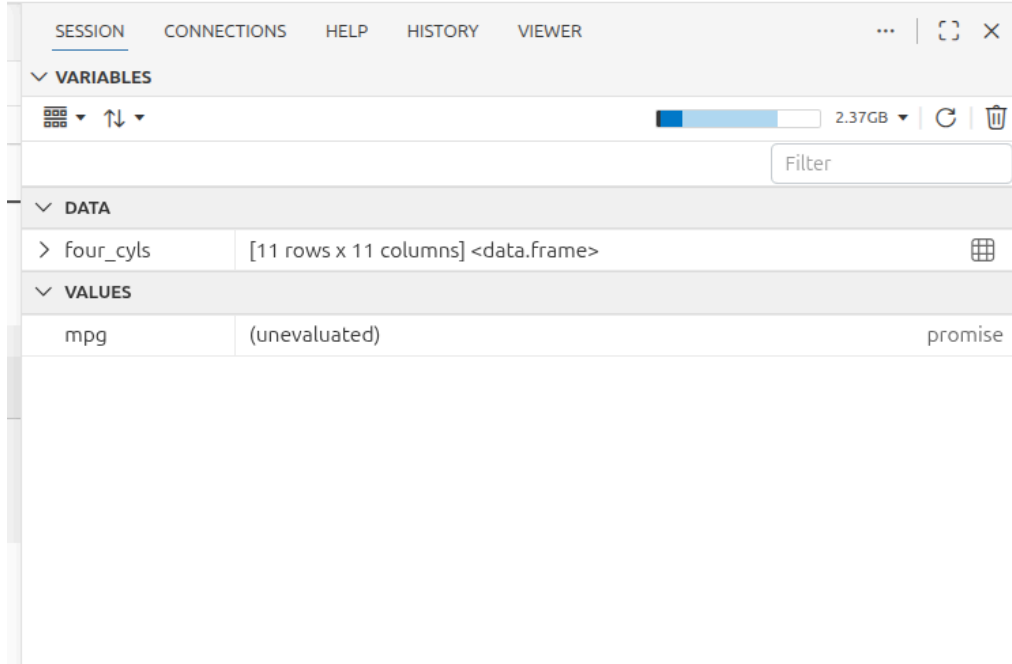
**Trouvez ces instructions dans votre carnet et compilez ce dernier.
Quelle est la différence en sortie?**

Assignment

<- assigne la sortie à droite à la variable de gauche

```
four_cyls <- filter(mtcars, cyl == 4)
```

Panneau variables



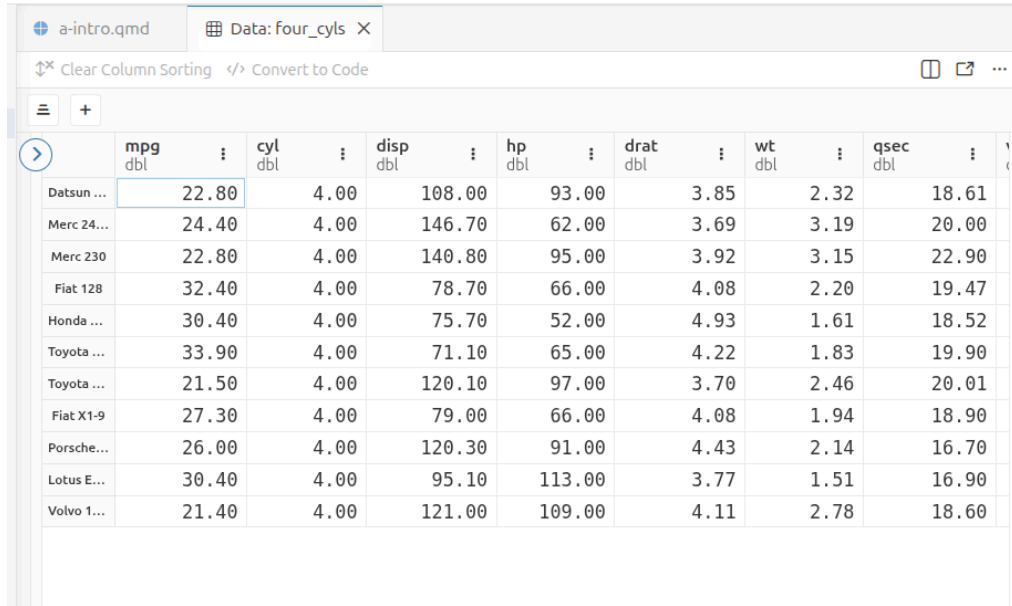
**Liste de toutes les
variables existantes**

À votre tour

Trouvez `four_cyls` dans le panneau Variables
Cliquez sur le nom `four_cyls`

Qu'est-ce qui arrive?

Visualisation



| | mpg
dbl | cyl
dbl | disp
dbl | hp
dbl | drat
dbl | wt
dbl | qsec
dbl |
|------------|------------|------------|-------------|-----------|-------------|-----------|-------------|
| Datsun ... | 22.80 | 4.00 | 108.00 | 93.00 | 3.85 | 2.32 | 18.61 |
| Merc 24... | 24.40 | 4.00 | 146.70 | 62.00 | 3.69 | 3.19 | 20.00 |
| Merc 230 | 22.80 | 4.00 | 140.80 | 95.00 | 3.92 | 3.15 | 22.90 |
| Fiat 128 | 32.40 | 4.00 | 78.70 | 66.00 | 4.08 | 2.20 | 19.47 |
| Honda ... | 30.40 | 4.00 | 75.70 | 52.00 | 4.93 | 1.61 | 18.52 |
| Toyota ... | 33.90 | 4.00 | 71.10 | 65.00 | 4.22 | 1.83 | 19.90 |
| Toyota ... | 21.50 | 4.00 | 120.10 | 97.00 | 3.70 | 2.46 | 20.01 |
| Fiat X1-9 | 27.30 | 4.00 | 79.00 | 66.00 | 4.08 | 1.94 | 18.90 |
| Porsche... | 26.00 | 4.00 | 120.30 | 91.00 | 4.43 | 2.14 | 16.70 |
| Lotus E... | 30.40 | 4.00 | 95.10 | 113.00 | 3.77 | 1.51 | 16.90 |
| Volvo 1... | 21.40 | 4.00 | 121.00 | 109.00 | 4.11 | 2.78 | 18.60 |

Cliquer sur un objet dans le panneau Variables l'ouvre dans un onglet de visualisation interactive.

Objets, classes et types

Vecteurs

Objet de base dans R

**Un vecteur est une liste ordonnée de valeurs de même type
(texte, nombres, etc.)**

Concaténez avec `c()`:

```
c(1, 4, 2, 5, 7)
```

Listes

Une liste est une collection d'objets

```
maliste <- list(elementun = 2:4, FALSE)
maliste
```

```
## $elementun
## [1] 2 3 4
##
## [[2]]
## [1] FALSE
```

Format de sortie fréquent pour les fonctions!

Éléments d'une liste

**Accéder à un élément
par nom avec \$**

```
maliste$elementun
```

**Accéder à un élément
par position avec [[]]**

```
maliste[[1]]
```

Base de données (`data.frame`)

Une liste avec dimensions

Chaque colonne est un vecteur

Variables de types potentiellement différents

Mais de même longueur

`tibble` (`tbl`): version `tidyverse`, plus sophistiquée

Types de données de base

| | | |
|-------------------------------------|---------------|---|
| Entier (<i>integer</i>) | Nombre entier | <code>c(1L, 2L, 3L, 4L); 1:4</code> |
| Double | Nombres | <code>c(1, 2.4, 3.14, 4)</code> |
| Caractère (<i>character</i>) | Texte | <code>c("1", "bleu", "plaisir", "monstre")</code> |
| Logique (<i>logical</i>) | Vrai ou faux | <code>c(TRUE, FALSE, TRUE, FALSE)</code> |

Utiliser la fonction `typeof()`
pour déterminer le type d'un vecteur.

Facteurs (factor, <fct>)

Vecteurs de variables catégorielles

Encodées comme entiers avec étiquettes
(étiquettes visible via `levels`)

Traitement spécial dans les modèles

```
facteur <- factor(x = c(1,2,1,3),  
                  labels = c("petit", "moyen", "grand"))  
facteur #méthode print
```

```
## [1] petit moyen petit grand  
## Levels: petit moyen grand
```

Valeurs manquantes

NA, NaN, NA_integer_, etc.

is.na() pour détecter

**summary() SUR data.frame
pour décompte par variable**

Fonctions

Fonctions égal actions

Arguments en intrant, résultats en sortie

Spécifier les arguments avec `nomarg =`, sinon en ordre

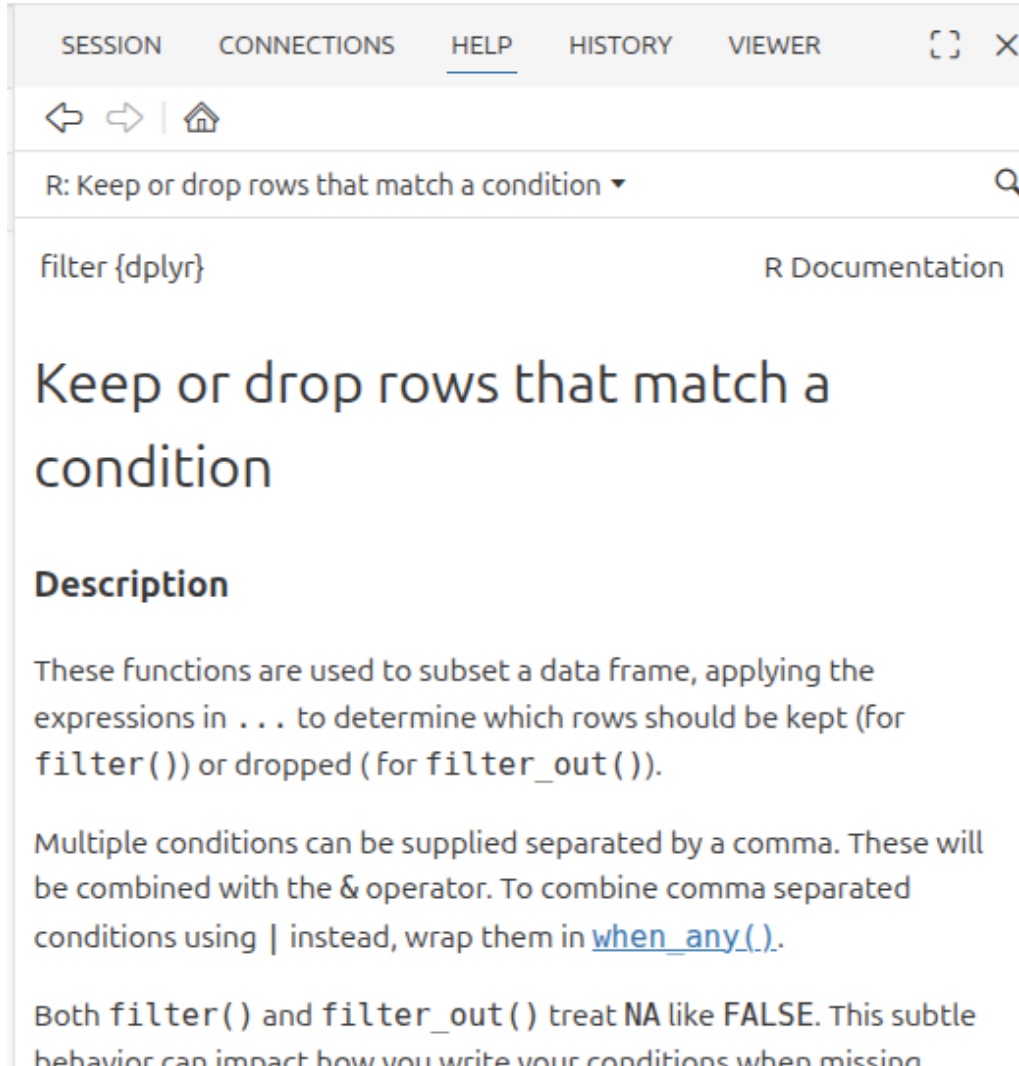
Aide

**Pour voir la page d'aide sur une fonction R,
tapez dans la console:**

```
?nom_fonction  
# Fonction pas dans l'environnement ou chaîne de caractères  
??"normal distribution"
```

(ou utilisez un moteur de recherche!)

Panneau d'aide



The screenshot shows the RStudio interface with the 'HELP' tab selected. The search bar contains 'R: Keep or drop rows that match a condition'. The search results show 'filter {dplyr}' from 'R Documentation'. The main content area displays the title 'Keep or drop rows that match a condition' and a 'Description' section. The description states: 'These functions are used to subset a data frame, applying the expressions in ... to determine which rows should be kept (for filter()) or dropped (for filter_out()). Multiple conditions can be supplied separated by a comma. These will be combined with the & operator. To combine comma separated conditions using | instead, wrap them in when_any(). Both filter() and filter_out() treat NA like FALSE. This subtle behavior can impact how you write your conditions when missing'.

SESSION CONNECTIONS HELP HISTORY VIEWER

← → | 🏠

R: Keep or drop rows that match a condition 🔍

filter {dplyr} R Documentation

Keep or drop rows that match a condition

Description

These functions are used to subset a data frame, applying the expressions in ... to determine which rows should be kept (for `filter()`) or dropped (for `filter_out()`).

Multiple conditions can be supplied separated by a comma. These will be combined with the `&` operator. To combine comma separated conditions using `|` instead, wrap them in `when_any()`.

Both `filter()` and `filter_out()` treat `NA` like `FALSE`. This subtle behavior can impact how you write your conditions when missing

**Ces fichiers contiennent
notamment des détails sur les
arguments des fonctions**

**Il y a souvent des
exemples à la fin**

À votre tour

Lisez l'aide pour la fonction `seq`

Ajouter un bloc qui utilise `seq()` pour créer un vecteur de nombres de 5 à 30, espacés par sauts de 5 (5, 10, 15, 20, ...)

02:00

```
seq(from = 5, to = 30, by = 5)
```

```
## [1]  5 10 15 20 25 30
```

Problèmes fréquents de syntaxe #1

Parenthèses ou accolades de fermeture manquante

```
mean(mtcars
```

"Oops problème

Problèmes fréquents de syntaxe #2

Ajouter des accolades quand ce n'est pas adéquat (et vice versa)

```
mean("mtcars")
```

```
## Warning in mean.default("mtcars"): l'argument n'est ni numérique, ni log  
## renvoi de NA
```

```
## [1] NA
```

Problèmes fréquents de syntaxe #3

Mauvais nom de fonction (R est sensible à la casse)

```
MEAN(mtcars)
```

```
## Error in `MEAN()`:
```

```
## ! impossible de trouver la fonction "MEAN"
```

À votre tour

Il y a trois blocs sous "Syntaxe fautive"

Compilez chacune, lisez le message d'erreur et essayez de corriger le code

Classes et méthodes

La classe d'un objet permet de définir des fonctions génériques (methods)

```
methods(class = "lm")
```

Autrement dit, le résultat dépend de la classe.

Méthodes fréquentes

impression
(print)

récapitulatif
(summary)

graphique
(plot)

Importer des données

Paquets pour l'importation de données



Données texte brutes

```
my_data <- read_csv("file.csv")
```



Chiffriers Excel

```
my_data <- read_excel("file.xlsx")
```



Données Stata, SPSS, et SAS

```
my_data <- read_stata("file.dta")
```